

React 19 Beta Version Updates





React 19 Beta is now available on npm! In this post, we'll give an overview of the new features in React 19, and how you can adopt them.

Note

This beta release is for libraries to prepare for React 19. App developers should upgrade to 18.3.0 and wait for React 19 stable as we work with libraries and make changes based on feedback.

 Like  Share  Repost  Save



What's new in React 19

Actions

A common use case in React apps is to perform a data mutation and then update state in response.

For example, when a user submits a form to change their name, you will make an API request, and then handle the response. In the past, you would need to handle pending states, errors, optimistic updates, and sequential requests manually.



For example, you could handle the pending and error state in useState:

```
// Before Actions
function UpdateName({}) {
  const [name, setName] = useState("");
  const [error, setError] = useState(null);
  const [isPending, setIsPending] = useState(false);

  const handleSubmit = async () => {
    setIsPending(true);
    const error = await updateName(name);
    setIsPending(false);
    if (error) {
      setError(error);
      return;
    }
    redirect("/path");
  };

  return (
    <div>
      <input value={name} onChange={(event) => setName(event.target.value)} />
      <button onClick={handleSubmit} disabled={isPending}>
        Update
      </button>
      {error && <p>{error}</p>}
    </div>
  );
}
```



Like



Share



Repost



Save



In React 19, we're adding support for using async functions in transitions to handle pending states, errors, forms, and optimistic updates automatically.

For example, you can use `useTransition` to handle the pending state for you:

```
// Using pending state from Actions
function UpdateName({}) {
  const [name, setName] = useState("");
  const [error, setError] = useState(null);
  const [isPending, startTransition] = useTransition();

  const handleSubmit = () => {
    startTransition(async () => {
      const error = await updateName(name);
      if (error) {
        setError(error);
        return;
      }
      redirect("/path");
    })
  };

  return (
    <div>
      <input value={name} onChange={(event) => setName(event.target.value)} />
      <button onClick={handleSubmit} disabled={isPending}>
        Update
      </button>
      {error && <p>{error}</p>}
    </div>
  );
}
```




The async transition will immediately set the `isPending` state to `true`, make the async request(s), and switch `isPending` to `false` after any transitions. This allows you to keep the current UI responsive and interactive while the data is changing.

Note

By convention, functions that use async transitions are called “Actions”.

Actions automatically manage submitting data for you:

- **Pending state:** Actions provide a pending state that starts at the beginning of a request and automatically resets when the final state update is committed.
- **Optimistic updates:** Actions support the new `useOptimistic` hook so you can show users instant feedback while the requests are submitting.
- **Error handling:** Actions provide error handling so you can display Error Boundaries when a request fails, and revert optimistic updates to their original value automatically.
- **Forms:** `<form>` elements now support passing functions to the `action` and `formAction` props. Passing functions to the `action` props use Actions by default and reset the form automatically after submission.

Building on top of Actions, React 19 introduces `useOptimistic` to manage optimistic updates, and a new hook `React.useActionState` to handle common cases for Actions. In react-dom we’re adding `<form>` Actions to manage forms automatically and `useFormStatus` to support the common cases for Actions in forms.



In React 19, the above example can be simplified to:

```
// Using <form> Actions and useActionState
function ChangeName({ name, setName }) {
  const [error, submitAction, isPending] = useActionState(
    async (previousState, formData) => {
      const error = await updateName(formData.get("name"));
      if (error) {
        return error;
      }
      redirect("/path");
      return null;
    },
    null,
  );

  return (
    <form action={submitAction}>
      <input type="text" name="name" />
      <button type="submit" disabled={isPending}>Update</button>
      {error && <p>{error}</p>}
    </form>
  );
}
```

In the next section, we'll break down each of the new Action features in React 19.

👍 Like ➦ Share ↺ Repost 📌 Save



New hook: useActionState

To make the common cases easier for Actions, we've added a new hook called useActionState:

```
const [error, submitAction, isPending] = useActionState(  
  async (previousState, newName) => {  
    const error = await updateName(newName);  
    if (error) {  
      // You can return any result of the action.  
      // Here, we return only the error.  
      return error;  
    }  
  
    // handle success  
    return null;  
  },  
  null,  
);
```

useActionState accepts a function (the “Action”), and returns a wrapped Action to call. This works because Actions compose. When the wrapped Action is called, useActionState will return the last result of the Action as data, and the pending state of the Action as pending.

Note

`React.useActionState` was previously called `ReactDOM.useFormState` in the Canary releases, but we've renamed it and deprecated `useFormState`.



Like



Share



Repost



Save



React DOM: <form> Actions

Actions are also integrated with React 19's new <form> features for react-dom. We've added support for passing functions as the action and formAction props of <form>, <input>, and <button> elements to automatically submit forms with Actions:

```
<form action={actionFunction}>
```

When a <form> Action succeeds, React will automatically reset the form for uncontrolled components. If you need to reset the <form> manually, you can call the new requestFormReset React DOM API.

For more information, see the react-dom docs for [<form>](#), [<input>](#), and [<button>](#).

👍 Like ➦ Share ↺ Repost 📌 Save



React DOM: New hook: useFormStatus

In design systems, it's common to write design components that need access to information about the <form> they're in, without drilling props down to the component. This can be done via Context, but to make the common case easier, we've added a new hook useFormStatus:

```
import {useFormStatus} from 'react-dom';

function DesignButton() {
  const {pending} = useFormStatus();
  return <button type="submit" disabled={pending} />
}
```

useFormStatus reads the status of the parent <form> as if the form was a Context provider.

👍 Like ➦ Share ↺ Repost 📌 Save



New hook: useOptimistic

Another common UI pattern when performing a data mutation is to show the final state optimistically while the async request is underway. In React 19, we're adding a new hook called `useOptimistic` to make this easier:

```
function ChangeName({currentName, onUpdateName}) {  
  const [optimisticName, setOptimisticName] = useOptimistic(currentName);  
  
  const submitAction = async formData => {  
    const newName = formData.get("name");  
    setOptimisticName(newName);  
    const updatedName = await updateName(newName);  
    onUpdateName(updatedName);  
  };  
  
  return (  
    <form action={submitAction}>  
      <p>Your name is: {optimisticName}</p>  
      <p>  
        <label>Change Name:</label>  
        <input  
          type="text"  
          name="name"  
          disabled={currentName !== optimisticName}  
        />  
      </p>  
    </form>  
  );  
}
```

The `useOptimistic` hook will immediately render the `optimisticName` while the `updateName` request is in progress. When the update finishes or errors, React will automatically switch back to the `currentName` value.

👍 Like

🔗 Share

↻ Repost

🔖 Save



New API: use

In React 19 we're introducing a new API to read resources in render: use.

For example, you can read a promise with use, and React will Suspend until the promise resolves:

```
import {use} from 'react';

function Comments({commentsPromise}) {
  // `use` will suspend until the promise resolves.
  const comments = use(commentsPromise);
  return comments.map(comment => <p key={comment.id}>{comment}</p>);
}

function Page({commentsPromise}) {
  // When `use` suspends in Comments,
  // this Suspense boundary will be shown.
  return (
    <Suspense fallback={<div>Loading...</div>}>
      <Comments commentsPromise={commentsPromise} />
    </Suspense>
  )
}
```




Note

`use` does not support promises created in render.

If you try to pass a promise created in render to `use`, React will warn:

```
Console
```

✖ A component was suspended by an uncached promise. Creating promises inside a Client Component or hook is not yet supported, except via a Suspense-compatible library or framework.

To fix, you need to pass a promise from a suspense powered library or framework that supports caching for promises. In the future we plan to ship features to make it easier to cache promises in render.

You can also read context with `use`, allowing you to read Context conditionally such as after early returns:

```
import {use} from 'react';
import ThemeContext from './ThemeContext'

function Heading({children}) {
  if (children == null) {
    return null;
  }

  // This would not work with useContext
  // because of the early return.
  const theme = use(ThemeContext);
  return (
    <h1 style={{color: theme.color}}>
      {children}
    </h1>
  );
}
```



Like



Share



Repost



Save



React Server Components

Server Components

Server Components are a new option that allows rendering components ahead of time, before bundling, in an environment separate from your client application or SSR server. This separate environment is the “server” in React Server Components. Server Components can run once at build time on your CI server, or they can be run for each request using a web server.

React 19 includes all of the React Server Components features included from the Canary channel. This means libraries that ship with Server Components can now target React 19 as a peer dependency with a `react-server` export condition for use in frameworks that support the Full-stack React Architecture.

👍 Like ➦ Share ↺ Repost 📌 Save



Server Actions

Server Actions allow Client Components to call async functions executed on the server.

When a Server Action is defined with the "use server" directive, your framework will automatically create a reference to the server function, and pass that reference to the Client Component. When that function is called on the client, React will send a request to the server to execute the function, and return the result.

Server Actions can be created in Server Components and passed as props to Client Components, or they can be imported and used in Client Components.

👍 Like ➦ Share ↺ Repost 📌 Save



Improvements in React 19

ref as a prop

Starting in React 19, you can now access ref as a prop for function components:

```
function MyInput({placeholder, ref}) {  
  return <input placeholder={placeholder} ref={ref} />  
}  
  
//...  
<MyInput ref={ref} />
```

New function components will no longer need forwardRef, and we will be publishing a codemod to automatically update your components to use the new ref prop. In future versions we will deprecate and remove forwardRef.



Diffs for hydration errors

We also improved error reporting for hydration errors in react-dom. For example, instead of logging multiple errors in DEV without any information about the mismatch:

```
Console
Warning: Text content did not match. Server: "Server" Client: "Client"
  at span
  at App
Warning: An error occurred during hydration. The server HTML was replaced with client content in
<div>.
Warning: Text content did not match. Server: "Server" Client: "Client"
  at span
  at App
Warning: An error occurred during hydration. The server HTML was replaced with client content in
<div>.
Uncaught Error: Text content does not match server-rendered HTML.
  at checkForUnmatchedText
  ...
```

We now log a single message with a diff of the mismatch:

```
Uncaught Error: Hydration failed because the server rendered HTML didn't match the client. As a
result this tree will be regenerated on the client. This can happen if an SSR-ed Client Component
used:

- A server/client branch if (typeof window !== 'undefined').
- Variable input such as Date.now() or Math.random() which changes each time it's called.
- Date formatting in a user's locale which doesn't match the server.
- External changing data without sending a snapshot of it along with the HTML.
- Invalid HTML tag nesting.

It can also happen if the client has a browser extension installed which messes with the HTML
before React loaded.

https://react.dev/link/hydration-mismatch

  <App>
    <span>
+   Client
-   Server
  at throwOnHydrationMismatch
```



<Context> as a provider

In React 19, you can render <Context> as a provider instead of <Context.Provider>:

```
const ThemeContext = createContext('');

function App({children}) {
  return (
    <ThemeContext value="dark">
      {children}
    </ThemeContext>
  );
}
```

New Context providers can use <Context> and we will be publishing a codemod to convert existing providers. In future versions we will deprecate <Context.Provider>.



Support for Document Metadata

In HTML, document metadata tags like `<title>`, `<link>`, and `<meta>` are reserved for placement in the `<head>` section of the document. In React, the component that decides what metadata is appropriate for the app may be very far from the place where you render the `<head>` or React does not render the `<head>` at all. In the past, these elements would need to be inserted manually in an effect, or by libraries like react-helmet, and required careful handling when server rendering a React application.

In React 19, we're adding support for rendering document metadata tags in components natively:

```
function BlogPost({post}) {
  return (
    <article>
      <h1>{post.title}</h1>
      <title>{post.title}</title>
      <meta name="author" content="Josh" />
      <link rel="author" href="https://twitter.com/joshcstory/" />
      <meta name="keywords" content={post.keywords} />
      <p>
        Eee equals em-see-squared...
      </p>
    </article>
  );
}
```

 Like  Share  Repost  Save



Repost this
to
help others

Comment down any topics if you want next !!

Follow for more content